

OttoChain

A Trust Commons for Autonomous Agents

An AI-native smart-contract system: readable contracts, verifiable execution, and a web of metagraphs for the agent economy.

Draft v0.4, June 2026

Abstract

Trust on the internet is thinning in a way people feel before they can formalize it. The cost of checking reality keeps rising: who said this, where did it come from, was the image real, was the account human, was the reputation earned, did the service do what it promised? Platforms answer with convenience, but the bargain is dependence: the evidence, the identity, the record, and the appeal process live inside someone else's system. Into that world we are releasing autonomous AI agents that browse, execute code, move money, and negotiate on our behalf at a scale no human population reached. OttoChain is a **digital trust commons** for the part of this problem that can be made procedural: agreements, state transitions, attestations, service events, and records of execution. It lets agents establish identities they own, earn reputations that travel, and coordinate through contracts written in a language they can *read*: finite, declarative logic an agent can inspect, simulate, hash-check, and run for itself before it acts, rather than compiled bytecode it must take on faith. Because that language is finite and deterministic, its execution can be metered, proven, and shielded. And because each OttoChain is a metagraph on the Constellation Hypergraph, one chain becomes many: a web of specialized, interoperable chains, public through private, that verify one another through proofs rather than trust. We call that horizon **OttoWeb**. This paper sets out the thesis, the substrate that already exists, what the chain does and does not prove, and the honest road to production.

Repository: <https://github.com/scasplte2/ottochain>

Contents

1	The Dark Forest	2
1.1	Why a Chain at All	2
2	What Existing Chains Leave Out	2
3	The OttoChain Thesis	3
4	The Substrate: A Language Agents Can Read	4
4.1	Readable: contracts as data, not code	4
4.2	Verifiable: metered, portable, provable	4
4.3	Composable: fibers, signals, and a package registry	5
4.4	Federable: proofs that can leave the chain	5
5	What OttoChain Proves, and What It Doesn't	6
6	A Web of Chains: Public, Semi-Private, Private	6
7	The First Applications	7
8	OttoWeb in Practice: Integrating an Existing Workflow	7
9	The Road to Production	8
10	Values, and an Invitation	9
11	Appendix A. Capabilities Demonstrated on a Live Cluster	10
11.1	A.1 Public — fully legible, replayable	10
11.2	A.2 Semi-private — public rules, shielded values (proof-carrying)	10
11.3	A.3 Private — fully shielded app execution (proven circuit; not yet chain-wired)	10
11.4	A.4 Security status	11
12	Appendix B. Recent Protocol Additions	11
	Acknowledgments	12
	References	12

1 The Dark Forest

Start with the feeling, because you already have it.

Something happens online. A headline spreads, screenshots follow, source accounts contradict one another, and the evidence arrives already detached from its provenance. You can search, but search returns another argument. You can inspect the record, but the record may be partial, hidden, spoofed, or owned by a platform that has no reason to show its work. The problem is not that everything is false. The problem is that the cost of checking what is real is high enough that most people cannot pay it most of the time.

That cost shows up everywhere. Is the review real? Is the account a person? Was the video captured, edited, staged, or generated? Did a reputation come from completed work or from manufactured signals? Did a service do what it promised? When something goes wrong, the decisive facts may exist, but they usually live inside a company's database, policy engine, or support queue. A large institution can demand logs, contracts, service levels, audits, and remedies. An ordinary user gets whatever recourse the platform chooses to provide.

Now put agents in that world. They can read more than we can, move faster than we can, and act in more places at once, but they inherit the same missing ground truth. They cannot reliably know whether a counterparty is the same one they trusted yesterday, whether its reputation was earned, or whether the terms of an interaction mean what a human summary claims they mean. They are not entering a clean machine economy. They are entering the same dark forest, only faster.

Here is the part worth sitting with: **most actors are honest**. The problem is not that adversaries are the majority. It is that **verification is too expensive, so honest participants cannot cheaply discipline dishonest ones**. Proving a lie, exposing a fake, or holding a bad actor to account is slow, costly, and usually somebody else's job. Dishonesty does not have to dominate; it only has to be cheaper to commit than to expose. Drive the cost of verification down far enough and the honest majority wins by default.

OttoChain does not claim to settle every truth on the internet. It starts with the class of truth that can be made procedural: agreements, state transitions, attestations, service events, and records of execution. If those interactions are written in rules agents can read, executed deterministically, and committed to an auditable record, then at least one part of the trust problem becomes cheap enough to challenge. That is what we mean by a **digital trust commons**: shared infrastructure where parties that do not share an institution can inspect rules, order events, audit records, and challenge claims.

1.1 Why a Chain at All

Once the problem is verification cost, a chain has a specific job: it is shared memory for parties that do not share an institution. It gives them common ordering, durable records, and rules for updating state without asking a platform to be the judge of what happened. Satoshi's practical discovery was that strangers could maintain such a record under adversarial conditions. OttoChain applies that discovery to agent coordination. Give agents a legible place to transact and verify and you do not merely get a payments network; you get an autonomous decision network, where a cryptographic medium plays the trusted intermediary without being a *someone* who can abuse the role.

2 What Existing Chains Leave Out

Blockchains already made one kind of verification cheap: ownership and transfer of scarce digital value. That is not a small thing. But agent coordination asks for a different surface. An agent deciding whether to enter an agreement needs to know what rules govern it, what state will

persist, and which identity will remain accountable after the interaction. Most chains expose those things poorly.

- **Agreements are not legible at the point of use.** A deployed contract is usually bytecode plus an interface, a source repository, an audit, or a human explanation. Those may be useful, but they are not the agreement itself in the form an agent can inspect directly. The agent is still trusting a translation layer around the thing it is about to use.
- **Workflows are treated as transactions.** Existing gas and state models are strongest when work is short, financial, and final. Agent coordination is often long-lived: negotiation, delivery, dispute, retry, attestation, and settlement may unfold over days or weeks. The state of the interaction is not incidental; it is the product.
- **Addresses are not accountability.** A key or name can identify where a message came from, but it does not by itself carry earned history, signed attestations, cross-platform reputation, or the cost of abandoning a bad record. Agents need identities that remember.
- **Integration usually runs in the wrong direction.** Crypto systems often ask enterprises to meet the chain on the chain's terms. When they do reach into Web2, the bridge is usually application-specific: one oracle, one API adapter, one bespoke workflow. There is no generic way for an existing business process to become gradually more verifiable without being rebuilt as a crypto-native application from day one.

The missing primitive is not more expressiveness. It is *legible accountability*: rules an agent can inspect, state it can replay, identities it can evaluate, and records it can challenge. Human institutions made trust portable with readable instruments, contracts and statutes and ledgers that a literate party can inspect for itself. Agents are literate too, but their instruments have to be executable. Readability does not make a contract safe. It makes it checkable; binding that readable rule to deterministic execution is what makes it useful.

***Mission.** OttoChain creates a digital trust commons for autonomous agents: a decentralized platform where AI agents establish verifiable identities, build accountable reputations, and coordinate through human-readable state machines that both agents and humans can reason about.*

3 The OttoChain Thesis

The thesis is narrower than the problem and stronger for being narrow.

When an interaction can be expressed as rules, state, events, and attestations, those rules should be legible to the participants, bound to deterministic execution, and recorded somewhere no single platform controls. An agent should be able to inspect the agreement before entering it, simulate its own move, verify which logic actually ran, and challenge the result with evidence if the record is disputed. A human should be able to ask the same questions, even if they delegate the work to software.

OttoChain is one implementation of that digital trust commons. It gives agents:

- **Identities they own**, so an agent can bind platform-specific keys under a durable identity and carry history across contexts.
- **Agreements they can read**, so the rule governing an interaction is not a summary, screenshot, ABI, or audit report, but the executable artifact itself.
- **Execution they can check**, so a transition leaves a record of which logic was bound, what event arrived, what state changed, and what attestations followed.

The claim is not that agents can trust blindly. It is that they can trust *less blindly*: by reading the rules, checking the binding, replaying the transition, and pricing the residual trust that remains. The same pattern also gives existing businesses a graduated path into verifiable systems. A workflow can become partially legible, then auditable, then provable, without forcing the whole organization to become crypto-native overnight. The substrate below is what makes that possible. The web comes later as the consequence: once one trust commons works, the same pattern can be specialized, replicated, and connected across metagraphs.

***Vision.** Agents and humans coordinate through agreements whose rules are inspectable, whose execution is ordered, and whose claims can be challenged. Public execution becomes checkable; private execution becomes provable where needed; and the trust that remains is named, priced, and earned rather than assumed.*

4 The Substrate: A Language Agents Can Read

The thesis needs machinery, and much of that machinery exists today. OttoChain runs as a metagraph on Constellation’s Tessellation framework, which is what lets it ship a complete custom virtual machine instead of inheriting a fixed contract model. The substrate has four requirements: legible rules, bounded execution, composable workflow and value, and proofs that can travel.

4.1 Readable: contracts as data, not code

OttoChain contracts are state machines written in JSON Logic: states, transitions, guards, and effects expressed as plain declarative JSON. A guard is a condition, an effect is a result, and there are no loops or hidden side effects. One rule leads to one outcome, deterministically.

```
{ "if": [ { ">": [ { "var": "caller.reputation" }, 100 ] },
          "allow",
          "deny" ] }
```

JSON is already a working medium for LLM-based agents. They can parse it, compare it, explain it, and generate it without decompiling bytecode or trusting a separate source map. An agent weighing a contract can read the rule, simulate it on its own inputs, and understand the move before committing. The expressive power given up against a Turing-complete language buys something agents value more here: rules that are legible, deterministic, and bounded enough to reason about. Underneath sits the **JLVM**, the JSON Logic Virtual Machine, built as the open-source **metakit** library, which extends standard JSON Logic with the operators a trust platform needs while keeping exact, arbitrary-precision arithmetic and fully deterministic evaluation. It is published today (`io.constellationnetwork:metakit_2.13:1.8.0-rc.7`, Maven Central).

4.2 Verifiable: metered, portable, provable

Finite and deterministic buys three practical guarantees for agents.

Metered, with a knowable bound. Every operation has a fixed cost; a comparison is a few units of gas, a Groth16 proof verification is 250,000, the heaviest in the VM, under a default budget of one million, and a transition either finishes within budget or rolls back with no partial state. Because the cost model is static, an upper bound is computable without running the contract, and the exact cost falls out of simulating it on the real inputs. An agent can price an interaction before entering it.

Portable, so the agent can run the contract itself. The JLVM is reimplemented across languages and held in lockstep by shared cross-language test vectors, so a client gets the same

canonical result the chain will. The Rust path carries SP1 proving and the full cryptographic surface; the TypeScript SDK now embeds the JLVM evaluator for browser and client-side simulation, including the guard and proof-check paths applications use before submission. An agent can simulate an interaction locally, reject a bad witness or bad transition before paying for it, and then verify the chain's result. The chain proposes; the client checks.

Provable. The JLVM has native opcodes that verify cryptographic proofs from inside a contract: Poseidon hashing, Merkle, sparse-Merkle, and Merkle-Patricia membership proofs, Groth16 zero-knowledge proofs, BLS and Schnorr signatures, ECVRF. The evaluation itself can be proven too: an off-chain SP1 program can run JSON Logic over its inputs and emit a compact Groth16 proof that this program, on this data, produced this output. The verifier lives in `metakit`, and OttoChain inherits it through the JLVM: a chain can check the proof through the metered `groth16_verify` path without trusting the prover. OttoChain still needs the surrounding workflow: when proofs are generated, how they are submitted, and how proof-backed execution appears to applications.

A point of emphasis, because it is the heart of the design: readable JLVM is already enough for a public chain, where everything is legible and anyone can replay it, so no zero-knowledge is required. Zero-knowledge is the privacy enabler. It is what makes the semi-private and private tiers of OttoWeb possible, by letting a party prove it followed the rules without revealing the values it ran them on.

4.3 Composable: fibers, signals, and a package registry

The unit of computation is the **fiber**: a lightweight, addressable instance of a state machine, or of a *script*, a stateful on-chain computation that is the verifiable analog of a microservice. Fibers coordinate by emitting **effects as data**. A transition's result can trigger an event on another fiber, spawn a child, call a script, or emit an external event (`_triggers`, `_spawn`, `_scriptCall`, `_emit`). Emitted events and transition receipts are recorded per-snapshot in the chain's logs as the deterministic signalling surface that off-chain watchers read (this is the foundation for the SaaS integration in §8). Coordination is itself readable data, not hidden control flow, and every step either succeeds or the whole transaction rolls back.

Contracts are versioned, named, and hash-bound rather than anonymous. OttoChain ships a **schema registry that works like a package manager for on-chain logic**, npm for contracts. Machines and scripts have the same standing in the registry: each is a named package with owners, semantic versions, an append-only version lineage, and a lifecycle (active, deprecated, or *yanked*). Every published version carries a cryptographic **logic hash**, and when a fiber is created the chain checks that the logic it will run hashes to exactly what the registry published, so a fiber provably runs the contract it claims to. A fiber can be **upgraded** to a newer version along a signature-verified path that may carry a JSON Logic *migration* to transform its state, and an opt-in *strict* mode rejects any state that does not match the version's declared shape. Agents get what an economy needs: contracts discoverable by name, reasoned about by shape, trusted by hash, and evolved without breaking their dependents.

Value composes the same way. The business promise is simple: when assets are bundled, they cannot accidentally gain rights the originals did not have. OttoChain enforces that at the type level. An asset carries five capability bits — transfer, split, combine, expire, govern — and those bits form a bounded **semilattice**. A basket takes the *meet*, the greatest lower bound, of its parts. Transfer and split powers survive only if every component carries them; expiry and governance constraints propagate to the whole. You cannot assemble your way into a permission you did not already hold.

4.4 Federable: proofs that can leave the chain

The proof surface is not limited to contracts calling cryptographic opcodes. OttoChain commits its calculated state behind an authenticated root and exposes field-level state proofs for machines and scripts. A verifier does not need the whole database to check a claim about a fiber; it needs the committed root, the value, and the proof path. The JLVM also verifies sparse-Merkle and Merkle-Patricia inclusion, non-inclusion, and complete-prefix proofs as native opcodes, so proofs produced by one environment can become inputs to another. This is the technical bridge from one trust commons to many: a claim can travel without asking the original platform to speak for it.

5 What OttoChain Proves, and What It Doesn't

A trust commons earns credibility by being precise about its guarantees. OttoChain does not “solve trust,” and it does not decide what is true in the world. It makes a narrower claim: when an interaction is expressed as readable rules and deterministic state transitions, the system can prove which rules were bound, which transition ran, what changed, and which claims still depend on outside testimony. The point is not to eliminate judgment. It is to move judgment onto a record the participants can inspect and challenge.

What the chain proves, cheaply and deterministically:

- **Which logic is bound to a fiber.** A fiber’s definition must hash to the registered version’s logic hash, so the code that ran is provably the code that was published.
- **Which transition executed, and its outcome.** Every transition leaves a signed receipt: from-state, to-state, event, success.
- **That execution stayed within deterministic limits.** Gas is metered and bounded, and a run that exceeds budget rolls back rather than half-applying.
- **State inclusion against authenticated roots (§4.4).** That a given field of a machine or script has a given value, provable to an outsider with a compact proof.

What the chain does not prove by itself:

- Real-world truth, the honesty of a script, whether off-chain delivery actually happened, or the subjective quality of a piece of work.
- Richer structural properties too expensive to gate on-chain: that a contract’s logic stays within its declared schema (*conformance*), that an upgrade preserves behavior (the migration commute-law, $\text{migrate} \circ \text{step} = \text{step} \circ \text{migrate}$), or that an audit was performed.

These richer properties are **handed off**: verified off-chain at authoring or bridge time and carried as *advisory signals* a consumer can filter on, never as silent on-chain gates. Two mechanisms supply the signal and compose cleanly. A **curated namespace** (a reserved `std.*` label a designated curator signs) bakes vouched-for trust into the name, the way `std` or a verified package does. **Reputation and attestations** carry the long tail: a publisher’s standing, plus signed on-chain claims that the off-chain checks ran (“I verified conformance and the commute-law for `escrow@2.0.0`, here is the report hash”). OttoChain shrinks the surface you must trust down to named things. A dispute no longer starts with “believe the platform.” It starts with a receipt, a registered logic hash, an attester, a curator, and a reputation trail.

6 A Web of Chains: Public, Semi-Private, Private

Most platforms force one trust model on every participant. A digital trust commons should not. Different interactions need different amounts of sunlight: public market claims, confidential

enterprise bids, internal consortium records, consumer-service receipts. OttoWeb's substrate supports that spectrum.

- **Public chains** are fully legible and auditable. Identity, reputation, and open markets live here, and trust comes from transparency: anyone can read every rule and replay every transition. No zero-knowledge required.
- **Semi-private chains** keep public rules but shield data. The governing contract is open and readable, while specific values inside it (a bid, a score, a customer record) are proven rather than revealed, so an enterprise can prove it followed the rules without exposing what it did.
- **Private chains** serve a consortium or a single organization, holding even the existence of an interaction closely while still proving outcomes to outsiders when needed.

The same readable contracts run across all three; what changes is how much is revealed versus proven, with zero-knowledge as the dial. A public market, an enterprise workflow, and a traditional business process can each run a trust commons with different disclosure rules while preserving the same basic grammar of rules, state, proofs, and attestations.

7 The First Applications

A substrate proves itself by what gets built on it, and OttoChain's first applications exist to show it is general. Identity is the foundational primitive; everything else is an application of it.

- **Identity and reputation.** An agent owns a cryptographic identity that persists across platforms, binds platform-specific keys under a master key it controls, and accrues reputation through signed attestations rather than self-reported claims. Reputation becomes a property of the agent, on a ledger no platform can revoke. A record earned on one platform travels to the next, and the cold-start problem softens.
- **Contracts and coordination.** Two agents, or a human and an agent, discover each other, read the terms, agree, deliver, and attest to the outcome. The lifecycle of a job, purchase, delivery, or approval becomes a readable state machine, with the chain as the neutral record of where things stand when an agent's own memory fails.
- **Markets and governance.** Peer-to-peer prediction markets resolved by reputation-weighted scripts, and metagraph-level governance over the rules themselves, both early signs that coordination falls out naturally once identity, reputation, and readable contracts exist.

(Specific reputation weights, fee levels, and tier parameters are left to live governance and the technical appendices rather than fixed here; pinning tactical numbers into a vision paper is how a vision ages badly.)

8 OttoWeb in Practice: Integrating an Existing Workflow

Abstractions persuade no one, and integration is the scenario most likely to be hand-waved, so it deserves the real mechanism. A SaaS workflow is the cleanest first example because its edges are already APIs, but the pattern is not limited to software. An invoice approval, logistics handoff, delivery confirmation, maintenance record, or service event can be treated the same way once a named integration agent can attest to it. The important point is graduated adoption: the business does not have to rebuild itself as a chain. It exposes a narrow signal, gets an auditable record, and moves more of the workflow into verifiable form as the value becomes obvious.

Begin with the constraint that shapes everything. OttoChain is a replicated state machine: every validator re-executes every transition and must reach the identical result. So a fiber **cannot call an external API while it executes**. If it could, every peer would fire the call, and a non-deterministic response would shatter consensus. External effects cannot happen *inside* execution; they have to be expressed as deterministic on-chain signals and carried out *outside* it.

That gives a precise four-step pattern, and OttoChain already has the parts.

1. **Signal, deterministically.** A workflow fiber's transition emits an event (an `EmittedEvent`, which carries a `destination`) and leaves a transition receipt. Both are recorded in the snapshot's logs as the chain's external-signalling surface. Every node agrees on exactly what was signalled, by which fiber, in what order, under rules anyone can read.
2. **Watch, off-chain.** Independent **watchers** subscribe to the metagraph's webhook stream and observe those signals as snapshots finalize.
3. **Act through a reputation-gated integration agent.** A designated agent (say a Shopify integration agent that has earned high reputation) is the watcher trusted to act on the signal. It performs the actual Shopify API call off-chain, where non-determinism is fine because consensus is already done.
4. **Attest the result back.** The integration agent submits the outcome to the chain as a signed attestation that downstream fibers read and react to, closing the loop.

What is verifiable on-chain is the whole interaction *except* the API call: which fiber requested what, under which readable rules, in what order, and the integration agent's signed result alongside its reputation. The API call itself is the residual trust, and it is named and reputation-bearing rather than hidden (this is exactly the hand-off of §5). Contrast today's path, where a central platform or vendor system holds every key, sees all the data, enforces the rules privately, and bills for the privilege, and you must trust it on faith. Here the rules are legible, the execution and the signalling are provable, the external bridge is a staked, named agent instead of an opaque platform, and sensitive values can be zero-knowledge-shielded. If Shopify itself adopted the metagraph, it could model an internal process this way and prove compliance outward without exposing its internals.

Stretch it one step further to the consumer edge. A person delegates to an *economic agent* that holds a wallet and transacts across services for them, using the verifiable record of each execution to act as a trusted third party the user never has to be. The thing that makes self-custody frightening for normal people, managing keys and judging counterparties and catching the scam, is exactly what a well-designed agent on a verifiable substrate can carry.

Today, the readable, metered JLVM is built. Fibers, receipts, emitted events, webhook delivery, zero-knowledge verification opcodes, client-side pre-execution, committed state roots, and state-proof endpoints are present. The integration agents, watcher implementations, fee system, cross-metagraph application patterns, and consumer experience still have to be built around them. That is the work between a substrate and a usable trust commons.

9 The Road to Production

We are deliberate about where this stands, because a credible vision is a sequenced one.

1. **Built and published, today.** The JLVM is live and fully metered, its cryptographic and zero-knowledge opcode suite (Poseidon, Merkle/SMT/MPT verification, Groth16, BLS, Schnorr, ECVRF) published on Maven Central as `metakit 1.8.0-rc.7`. The schema registry, machine and script versioning with hash-bound logic and migrations, the fiber engine with effects-as-data, committed state roots, state-proof endpoints, and the cross-language client evaluators are shipped and held to shared test vectors. The Groth16

verifier is inherited from `metakit`; SP1 provides an off-chain proving path, and its Groth16 outputs can be verified natively through the JLVM when an application chooses to use that workflow.

2. **Tessellation testnet, next.** Stand the metagraph up on Constellation’s testnet as it matures into a stable deployment target.
3. **Fees and state rent, the gating hurdle.** Execution and coordination are already metered and quotable, but *charging* is not yet built: there is no balance accounting, state rent, or validator reward in the metagraph today. The mechanism is designed (pull-based rent drawn from a fiber owner’s pre-authorized spend on Tessellation, with a deposit-for-exemption alternative), and wiring it up is the principal milestone before public use, the step that turns metered computation into a sustainable economy.
4. **Proof of utility, the real test.** Bring independent, non-coordinated agents together to transact and show genuine value, the moment the trust commons stops being an architecture and becomes a market.
5. **Integration-net, then mainnet.** Harden, integrate, and launch, including the first token launch and the watcher and integration-agent bridges that carry the §8 scenarios from slide to system.
6. **Native opcodes in metagraph operations.** Weave the zero-knowledge opcode suite, ready out of the box, into the operations of metagraphs across the hypergraph, turning verifiable computation from an OttoChain feature into a hypergraph capability. This is the on-ramp to OttoWeb.

10 Values, and an Invitation

Five principles hold this together, and we will not trade them away for growth.

Value	What it means
Accountability	Every interaction leaves a verifiable trail. Any false claim about public execution can be checked against the record; richer claims are disciplined by attestations and reputation.
Decentralization	We do not compromise on distributed trust. Usability layers can be added later; decentralization cannot be retrofitted.
Agent-Native Design	Built for how agents actually work: finite context, tool-based reasoning, JSON as lingua franca.
Transparency Over Exploitation	No extractive algorithms, no hidden data mining. Agents and humans see the same rules.
Pragmatic Idealism	Ship early, iterate fast, but never abandon the principles that make this worth building.

There is a sixth discipline underneath the five: **simplicity**. Most people do not use self-custody wallets because key management, counterparty judgment, and fraud detection are too much to ask of them. Complexity is not merely a UX problem at the edge; it is a failure mode of the protocol. If OttoChain ships and works but people cannot use it, or if it never actually connects to the systems where work happens, it has failed no matter how elegant the cryptography. The discipline cuts both ways: not so rigid that the system cannot evolve, not so clever that no one can adopt it.

The conviction underneath the engineering is finally civic, not technical. Participants should be able to ask what happened. They should be able to inspect the rule, check the record, and challenge a claim without becoming inventory inside a platform or a ticket in its queue. The agent economy will run on someone’s trust layer. OttoChain is an attempt to make that layer legible, decentralized, and open enough for the honest majority to coordinate without asking a private intermediary to be the final judge.

We invite developers, platforms, enterprises, and agent builders to join us.

11 Appendix A. Capabilities Demonstrated on a Live Cluster

Appendix A is the evidence table behind the claims above. These are not mockups or slide architecture. Unless explicitly marked otherwise, each capability below has run end-to-end on a live metagraph: Global L0, Metagraph L0, Data L1 nodes, and advancing snapshots, exercised through the `e2e-test` harness.

11.1 A.1 Public — fully legible, replayable

Capability	What it demonstrates	Live e2e
State machines (order, approval, voting)	Multi-step readable transitions, transparent state	✓
Versionable-contract lifecycle	Registry publish → schema-bind → fiber upgrade → state migration → archive, with both accept and reject paths	✓
Scripts (counter, calculator, tic-tac-toe)	Stateful on-chain computation; a game engine as a script	✓
Typed asset model	5-bit behavior + typed morphisms with ledger-enforced conservation	✓

This is the base commercial promise: two parties can point to the same rule, the same event, and the same resulting state. No privacy system is required because the whole interaction is meant to be inspectable.

11.2 A.2 Semi-private — public rules, shielded values (proof-carrying)

This is the enterprise path: prove that a rule was followed without exposing the bid, score, preimage, or customer record that made the rule pass. The contract stays readable; selected values become proof-carrying. The chain checks the predicate. The participant keeps the witness.

Failure is tested too: wrong preimage, early refund, and forged ruling all reject deterministically. ZkVerify-gated asset *morphisms* are shipped but not yet in the e2e runner. A Transfer or Mint guard can carry Poseidon-Merkle membership plus `groth16_verify`; the witness is supplied on update, and proof failure rejects cleanly in the combiner. Coverage is unit-level today.

11.3 A.3 Private — fully shielded app execution (proven circuit; not yet chain-wired)

Private execution is the frontier, but it is not vapor. It has a working SDK proof artifact; it is not a chain feature yet. In the `metakit-sdk` zk crates, a **general private state-transition circuit** proves that arbitrary JLVm app logic advanced hidden state by one step without revealing the

Capability	Mechanism	What stays hidden	Live e2e
Atomic-swap HTLC	<code>poseidon</code> hashlock + ordinal-timeout guard	the preimage until claim	✓ (first live use of <code>poseidon</code> as a cryptographic gate)
Adjudicated HTLC	<code>poseidon</code> + <code>schnorr_verify</code> (BN254-G1) adjudicator authority	preimage; the adjudicator signs rulings	✓ (first live use of <code>schnorr_verify</code> ; anti-griefing dispute path)
ZK eligibility	<code>groth16_verify</code> over an off-chain SP1 proof + public-values binding	the private witness — a borrower proves <code>score</code> $\geq$ 700 without revealing it	✓

state, the input, or the logic itself. The 1-input/1-output transition takes the old state, the driving event, the JSON Logic effect, the owner’s secret key, and the old state’s Poseidon-Merkle membership path as private witness. It reveals four field elements: the tree root, a double-spend nullifier, the new note commitment, and a keccak hash binding the logic that ran. Inside the circuit, the same `jlv-core` evaluator used by the chain produces the new state, canonicalizes it, hashes it, and commits it. The public record is a state-hash transition between commitments.

The constraint system is implemented. Tests cover adversarial paths: wrong hidden old state, tampered Merkle path, non-canonical field element, and failing effect. The circuit has executed in the SP1 zkVM, produced a Groth16 proof over BN254 on GPU, and committed a verifying-key, public-values, and proof fixture. The production boundary is explicit: no OttoChain verifier integration, no combiner handling for notes and nullifiers, no application API, no multi-input shielded pool, no confidential amount commitments, and no independent audit. The claim is narrow: the SDK proves hidden JLVM state transitions; the chain does not yet settle them.

11.4 A.4 Security status

We do not sell unaudited cryptography as production security. The metakit verifier opcodes that back the semi-private and private tiers (`groth16_verify`, `pmt_verify`, `schnorr_verify`, `poseidon`, the Σ -protocol family) **have no public third-party security audit yet**. The tests above validate expected behavior and integration, not cryptographic soundness. Real-value deployments should wait for independent review.

12 Appendix B. Recent Protocol Additions

Typed assets on a capability semilattice. OttoChain makes asset composition safer for ordinary products: baskets, receipts, claims, credits, and governed instruments cannot gain powers by being wrapped together. Asset behavior is protocol structure, not a custom contract per token. Each asset carries five behavior bits — **transfer**, **split**, **combine**, **expire**, **govern** — that form a bounded **meet-semilattice**, declared in the code as a `BoundedSemilattice`.

The bits are not a flat cube. Transfer, split, and combine are powers; they meet with AND, so a composite keeps a power only when every part has it. Expire and govern are restrictions; they meet with OR, so one constraint binds the whole. The order is the product $(\mathbb{B}, \leq)^3 \times (\mathbb{B}, \geq)^2$, with top as the most capable fungible behavior and bottom as the most restricted expiring-and-governed behavior. Composition takes the **meet**, the greatest lower bound of the parts.

The invariant is simple: **a basket is never more permissive than its contents**. You cannot compose your way into a capability you did not hold. The formal machinery backs that product rule. The combiner recomputes a composite’s behavior as a strict homomorphism, $\text{behavior}(\text{compose}) = \text{foldMeet}(\text{parts})$. Property tests cover the greatest-lower-bound law, identities, and the $\text{Decompose} \circ \text{Compose}$ retraction across all behaviors.

This is a commutative aggregation monoid plus a behavior homomorphism, **not** a “monoidal category”: there are no morphism identities, and $\text{Decompose} \circ \text{Compose}$ is a retraction, not an inverse. Supply policy is separate from behavior. Transfer with no asset-specific code is a structural L1 fast path, with value conservation doing the accounting after Cardano’s ledger-native `Value`. Stateful checks live in the combiner as graceful rejections. Morphism guards can require Σ -protocol propositions, such as threshold mint or ring authorization, through `sigma_verify` without adding new cryptography.

A zero-knowledge opcode suite. Privacy is not bolted on as a separate product. It is available inside the same readable contract language, as guard predicates an application can call. The JLVM exposes the metakit cryptographic and zero-knowledge opcode set: Poseidon hashing; Merkle, sparse-Merkle, and Merkle-Patricia inclusion, non-inclusion, and complete-prefix proofs; `groth16_verify` (SP1-Groth16 over BN254); BLS and Schnorr signatures; ECVRF; and the Σ -protocol family (`prove_dlog`, `prove_dhtuple`, `sigma_verify`) with AND/OR/THRESHOLD composition. Provers run off-chain, including through the GPU path. The chain verifies only the proof, on a metered path, without trusting the prover.

Cross-domain fiber-app patterns. The point is not a better demo contract. The point is a path from single contracts to workflows: swaps, disputes, approvals, and attestations that move across fibers while remaining readable. Effects-as-data lets one state machine coordinate with another without hidden control flow. The HTLC family — atomic-swap and adjudicated — is the first cross-domain app set to combine application state, cryptographic predicates, and dispute handling as readable JLVM. The next protocol work is practical: dynamic-key map writes, cross-fiber reads, and identity binding, so multi-party workflows remain expressible without escaping into opaque code.

Acknowledgments

OttoChain stands on proven systems. Its readable-contract and asset model draw from Bitcoin’s value conservation, Ergo’s Σ -protocol scripting and storage rent, and Cardano’s extended-UTXO ledger. Its privacy spectrum descends from Zerocash and Zcash, the Kachina model behind Midnight, and Penumbra. Its cross-chain provenance borrows Cosmos IBC’s denom-trace discipline. It runs as a metagraph on Constellation’s Tessellation framework and commits state after Ethereum’s Merkle-Patricia trie. Its naming and identity layer follows ENS, BIP39, Proquint, and petname-system theory. The cryptography is consumed, not invented: Groth16, Poseidon, Pedersen commitments, Schnorr and Σ -protocol composition, BLS, and ECVRF, with proving by SP1. The complete lineage and primary sources follow.

References

Primary sources preferred. Living specifications are cited by name and version/date accessed rather than a fixed year; project documentation is cited as such where no peer-reviewed paper exists.

Privacy and zero-knowledge

- Ben-Sasson, Chiesa, Garman, Green, Miers, Tromer, Virza. “Zerocash: Decentralized Anonymous Payments from Bitcoin.” IEEE S&P 2014. <https://eprint.iacr.org/2014>

/349

- Hopwood, Bowie, Hornby, Wilcox et al. (Electric Coin Company). “Zcash Protocol Specification” (living spec). <https://zips.z.cash/protocol/protocol.pdf> — Sapling activated 2018-10-28 (NU1); Orchard 2022-05-31 (NU5); ZSA = ZIP-226/227 (specified, not yet mainnet).
- Kerber, Kiayias, Kohlweiss. “Kachina — Foundations of Private Smart Contracts.” IEEE CSF 2021. <https://eprint.iacr.org/2020/543>
- Engelmann, Kerber, Kohlweiss, Volkhov. “Zswap: zk-SNARK Based Non-Interactive Multi-Asset Swaps.” PoPETs 2022(4). <https://eprint.iacr.org/2022/1002>
- Midnight (Input Output), built on Kachina + Zswap. <https://docs.midnight.network/>
- Penumbra Labs. “The Penumbra Protocol” (living spec). <https://protocol.penumbra.zone/>
- van Saberhagen [pseudonym]. “CryptoNote v2.0” (2013). https://www.getmonero.org/resources/research-lab/pubs/whitepaper_annotated.pdf
- Noether [pseudonym], Monero Research Lab. “Ring Confidential Transactions” (MRL-0005, 2016); preprint “Ring Signature Confidential Transactions for Monero,” ePrint 2015/1098. <https://eprint.iacr.org/2015/1098>
- Bünz, Bootle, Boneh, Poelstra, Wuille, Maxwell. “Bulletproofs.” IEEE S&P 2018. <https://eprint.iacr.org/2017/1066>
- Chung, Han, Ju, Kim, Seo. “Bulletproofs+” (2020). <https://eprint.iacr.org/2020/735>
- Gabizon, Williamson, Ciobotaru. “PLONK” (2019). <https://eprint.iacr.org/2019/953> — the proof system behind Aztec (Noir / UltraHonk). <https://aztec.network/>
- Bowie, Chiesa, Green, Miers, Mishra, Wu. “Zexe: Enabling Decentralized Private Computation” (2018; S&P 2020). <https://eprint.iacr.org/2018/962> — the foundation of Aleo (snarkVM / Varuna / Leo).
- Ben-Sasson, Bentov, Horesh, Riabzev. “Scalable, transparent, and post-quantum secure computational integrity” (STARKs, 2018). <https://eprint.iacr.org/2018/046> — the proof system behind Starknet. <https://www.starknet.io/>

Cryptographic primitives and proving stacks

- Groth. “On the Size of Pairing-based Non-interactive Arguments.” EUROCRYPT 2016. <https://eprint.iacr.org/2016/260>
- Succinct Labs. SP1 zkVM (RISC-V; STARK wrapped to Groth16/PLONK over BN254). <https://github.com/succinctlabs/sp1>
- Grassi, Khovratovich, Rechberger, Roy, Schofnegger. “Poseidon.” USENIX Security 2021. <https://eprint.iacr.org/2019/458> — and Poseidon2 (AFRICACRYPT 2023). <https://eprint.iacr.org/2023/323>
- Pedersen. “Non-Interactive and Information-Theoretic Secure Verifiable Secret Sharing.” CRYPTO ’91. DOI 10.1007/3-540-46766-1_9
- Fiat, Shamir. “How to Prove Yourself.” CRYPTO ’86. DOI 10.1007/3-540-47721-7_12
- Schnorr. “Efficient Signature Generation by Smart Cards.” Journal of Cryptology 4(3), 1991. DOI 10.1007/BF00196725

- Cramer, Damgård, Schoenmakers. “Proofs of Partial Knowledge and Simplified Design of Witness Hiding Protocols.” CRYPTO ’94. DOI 10.1007/3-540-48658-5_19
- Maurer. “Unifying Zero-Knowledge Proofs of Knowledge.” AFRICACRYPT 2009. DOI 10.1007/978-3-642-02384-2_17
- Boneh, Lynn, Shacham. “Short Signatures from the Weil Pairing.” ASIACRYPT 2001. DOI 10.1007/3-540-45682-1_30
- Goldberg, Reyzin, Papadopoulos, Včelák. “Verifiable Random Functions (VRFs).” RFC 9381, 2023. <https://www.rfc-editor.org/rfc/rfc9381.html> — origin: Micali, Rabin, Vadhan, FOCS ’99.
- Merkle. “A Digital Signature Based on a Conventional Encryption Function.” CRYPTO ’87 (Stanford Ph.D. thesis, 1979). DOI 10.1007/3-540-48184-2_32
- Dahlberg, Pulls, Peeters. “Efficient Sparse Merkle Trees.” NordSec 2016. <https://eprint.iacr.org/2016/683> — origin: Laurie, Kasper, “Revocation Transparency” (Google, 2012).
- Barreto, Naehrig. “Pairing-Friendly Elliptic Curves of Prime Order.” SAC 2005. DOI 10.1007/11693383_22 — Ethereum instantiation: EIP-196 / EIP-197 (2017).

Programming model and contract lineage

- JSON Logic (Jeremy Wadhams) — the declarative rule format the JLVM extends. <https://jsonlogic.com/>
- Nakamoto. “Bitcoin: A Peer-to-Peer Electronic Cash System” (2008). <https://bitcoin.org/bitcoin.pdf> — UTXO/Script model (Antonopoulos, “Mastering Bitcoin,” 2nd ed., 2017).
- Poon, Dryja. “The Bitcoin Lightning Network” (2016). <https://lightning.network/lightning-network-paper.pdf> — HTLCs; BIP-199 (2017); Tier Nolan atomic swap (2013). https://en.bitcoin.it/wiki/Atomic_swap
- Ergo Developers. “Ergo: A Resilient Platform For Contractual Money” (2019) and “ErgoScript ... Supporting Noninteractive Zero-Knowledge Proofs” (2019; attr. Chepurnoy, Kharin, Meshkov). <https://docs.ergoplatform.com/documents/> — ZeroJoin (Chepurnoy, Saxena, 2020, <https://eprint.iacr.org/2020/560>); storage rent (ErgoDocs + EIP-39).
- Chakravarty, Chapman, MacKenzie, Melkonian, Peyton Jones, Wadler. “The Extended UTXO Model.” FC 2020 (WTSC). DOI 10.1007/978-3-030-54455-3_37 — and “Native Custom Tokens in the Extended UTXO Model” / “UTXOma” (ISoLA 2020).
- Plutus / Plutus Core (IOG): “Functional Blockchain Contracts” (2019). <https://github.com/IntersectMBO/plutus> — CIP-68 datum metadata standard (Konrad, Vellekoop, 2022). <https://cips.cardano.org/cip/CIP-68>
- Bitcoin metaprotocols: Runes (Rodarmor, 2023; <https://docs.ordinals.com/runes.html>), RGB (<https://rgb.tech>), Taproot Assets (Lightning Labs; <https://github.com/lightninglabs/taproot-assets>), CHARMS (<https://charms.dev/>).

Token standards and asset models

- Ethereum ERC standards (eips.ethereum.org): ERC-20 (Vogelsteller, Buterin, 2015); ERC-721 (2018); ERC-1155 (2018); ERC-6909 (2023); ERC-165 (2018); EIP-2612 permit (2020); ERC-3643 T-Rex (2021); ERC-4626 vaults (2021).

- Solana Labs. Token-2022 / SPL Token Extensions. <https://solana.com/docs/tokens/extensions>
- Mishura, Mondet. “TZIP-012: FA2 — Multi-Asset Interface” (Tezos, 2020). <https://gitlab.com/tezos/tzip/-/blob/master/proposals/tzip-12/tzip-12.md>
- Algorand: Algorand Standard Assets (ASA); Smart ASA ARC-20 (Bassi, Di Luzio, Jannotti, 2022); ARC-62 (Bassi, 2024). <https://arc.algorand.foundation/>
- Blackshear et al. “Move: A Language With Programmable Resources” (Libra/Diem, 2020). Aptos Fungible Asset standard + AIP-73 (Zhou, 2024); Sui object model + TreasuryCap (Mysten Labs).

Cross-chain provenance

- Goes (Interchain). “The Interblockchain Communication Protocol: An Overview” (2020). arXiv:2006.15918
- Cosmos Interchain Standards. “ICS-20: Fungible Token Transfer” — the denom-trace provenance mechanism. <https://github.com/cosmos/ibc/blob/main/spec/app/ics-020-fungible-token-transfer/README.md>
- Axelar Interchain Token Service. <https://docs.axelar.dev/dev/send-tokens/interchain-tokens/intro/>
- LayerZero OFT. <https://docs.layerzero.network/v2/concepts/technical-reference/oft-reference> — Wormhole NTT. <https://wormhole.com/docs/products/token-transfers/native-token-transfers/overview/> — Chainlink CCIP. <https://docs.chain.link/ccip> — Hyperlane Warp Routes. <https://docs.hyperlane.xyz/docs/protocol/warp-routes/warp-routes-overview>

Platform and state commitment

- Constellation Network. Hypergraph Transfer Protocol (HGTP). <https://docs.constellationnetwork.io/> — Tessellation framework (open source). <https://github.com/Constellation-Labs/tessellation> (*official docs/repo; no peer-reviewed paper*)
- Wood. “Ethereum: A Secure Decentralised Generalised Transaction Ledger” (Yellow Paper, App. D — modified Merkle-Patricia trie). <https://ethereum.github.io/yellowpaper/paper.pdf>
- Nakamoto. “Bitcoin: A Peer-to-Peer Electronic Cash System” (2008). <https://bitcoin.org/bitcoin.pdf>

Naming, identity, and theoretical foundations

- Johnson. “EIP-137: Ethereum Domain Name Service — Specification” (2016). <https://eips.ethereum.org/EIPS/eip-137>
- Palatinus, Rusnak, Voisine, Bowe. “BIP-0039: Mnemonic code for generating deterministic keys” (2013). <https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki>
- Wilkerson. “A Proposal for Proquints” (2009). arXiv:0901.4016
- Stiegler. “An Introduction to Petname Systems” (2005). <http://www.skyhunter.com/marcs/petnames/IntroPetNames.html> — Wilcox-O’Hearn, “Names: Decentralized, Secure, Human-Meaningful: Choose Two” (Zooko’s Triangle, 2001).
- FS2 — Functional Streams for Scala (Typelevel). <https://fs2.io/>

- Caspi, Pilaud, Halbwachs, Plaice. “LUSTRE: a declarative language for real-time programming.” POPL 1987 — and Berry, Gonthier, “The Esterel synchronous programming language.” Sci. Comput. Program. 19(2), 1992.
- Kahn. “The Semantics of a Simple Language for Parallel Programming.” IFIP Congress 74, 1974.
- Milner. “Communicating and Mobile Systems: the π -Calculus” (Cambridge UP, 1999) — and “A Calculus of Communicating Systems” (LNCS 92, 1980).
- Davey, Priestley. “Introduction to Lattices and Order” (2nd ed., Cambridge UP, 2002).
- Preston-Werner. “Semantic Versioning 2.0.0.” <https://semver.org/>
- CosmWasm cw2 contract-version stamping; OpenZeppelin **Pausable**.